Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 2: Empirical Analysis of Comparison-Based and Non-Standard Sorting Algorithms

Author:
st. gr. FAF-241 Moisei Daniel
Verified:
asist. univ. Fiștic Cristofor

Chișinău - 2026

Contents

ALGORITHM ANALYSIS **2**
Objective 2
Tasks 2
Introduction 2

IMPLEMENTATION & SCENARIO ANALYSIS **3**
MergeSort (Basic) 3
MergeSort (Optimized) 6
QuickSort (Basic) 8
QuickSort (Optimized) 11
HeapSort (Basic) 14
HeapSort (Optimized) 17
PatienceSort (Basic) 20
PatienceSort (Optimized) 24

OVERALL RESULTS **28**

CONCLUSION **29**

REFERENCES **30**

ALGORITHM ANALYSIS

Objective

Study, implement, and analyze the performance of eight sorting variants: basic and optimized versions of MergeSort, QuickSort, HeapSort, and PatienceSort under different input conditions and data types.

Tasks:

1. Implement basic and optimized versions of MergeSort, QuickSort, HeapSort, and PatienceSort;
2. Analyze QuickSort partitioning: comparing Lomuto and Hoare schemes with optimized pivot selection;
3. Evaluate algorithms against Random, Sorted, Reversed, and Duplicate data;
4. Analyze execution time complexity for Best, Average, and Worst cases;
5. Test algorithm robustness against extreme cases (Negatives, Doubles);
6. Deduce conclusions based on empirical graphs and theoretical knowledge.

Introduction:

Sorting is one of the most fundamental operations in computer science. In this laboratory, we implement eight sorting variants to observe how actual performance varies based on the initial state of the data and implementation logic. Sorting algorithms are essential components of many real-world systems, influencing the efficiency of databases, search engines, and data processing pipelines. Although theoretical time complexity provides a mathematical performance bound, practical execution time is often affected by constant factors, memory access patterns, and input distribution. Therefore, an empirical evaluation is necessary to understand how different design choices impact performance under realistic conditions.

IMPLEMENTATION & SCENARIO ANALYSIS

MergeSort (Basic)

MergeSort is a stable, divide-and-conquer algorithm that guarantees performance by splitting data into halves and merging them.

Pseudocode:

Listing 1: MergeSort Recursive Structure

```
MergeSort(A, p, r):
    if p >= r:
        return
    mid = (p + r) // 2
    MergeSort(A, p, mid)
    MergeSort(A, mid + 1, r)
    Merge(A, p, mid, r)

Merge(A, p, mid, r):
    n1 = mid - p + 1
    n2 = r - mid

    create arrays L[0..n1-1]
    create arrays R[0..n2-1]

    for i = 0 to n1 - 1:
        L[i] = A[p + i]
    for j = 0 to n2 - 1:
        R[j] = A[mid + 1 + j]

    i = 0
    j = 0
    k = p

    while i < n1 and j < n2:
        if L[i] <= R[j]:
            A[k] = L[i]
            i = i + 1
        else:
            A[k] = R[j]
            j = j + 1
        k = k + 1

    while i < n1:
        A[k] = L[i]
        i = i + 1
        k = k + 1
```

```
    while j < n2:
        A[k] = R[j]
        j = j + 1
        k = k + 1
```

Implementation:

```
public static class MergeSortBasic
{
    public static void Sort(int[] arr)
    {
        if (arr == null || arr.Length < 2)
        {
            return;
        }

        Sort(arr, 0, arr.Length - 1);
    }

    private static void Sort(int[] arr, int left, int right)
    {
        if (left < right)
        {
            int mid = left + (right - left) / 2;
            Sort(arr, left, mid);
            Sort(arr, mid + 1, right);
            Merge(arr, left, mid, right);
        }
    }

    private static void Merge(int[] arr, int l, int m, int r)
    {
        int[] leftArr = arr[l..(m + 1)];
        int[] rightArr = arr[(m + 1)..(r + 1)];
        int i = 0;
        int j = 0;
        int k = l;

        while (i < leftArr.Length && j < rightArr.Length)
        {
            arr[k++] = leftArr[i] <= rightArr[j] ? leftArr[i++] : rightArr[j++];
        }

        while (i < leftArr.Length)
        {
            arr[k++] = leftArr[i++];
        }

        while (j < rightArr.Length)
        {
            arr[k++] = rightArr[j++];
        }
    }
}
```

Figure 1: MergeSort implementation in C#

Complexity Facts & Performance Defects:

- **Complexity:** $O(n \log n)$ for Best, Average, and Worst cases. Space complexity is $O(n)$.
- **Why basic is slower:** The fundamental defect of the basic version is dynamic memory allocation. Instantiating new left and right auxiliary arrays during every recursive ‘Merge‘ call creates massive heap allocation overhead and poor CPU cache utilization.

Results:

```
--- MERGESORT (BASIC) ---
Random Array (15000):
  Average: 2.265 ms  [ 2.269, 2.254, 2.270 ]
  Sorted: ✓ Yes
Sorted Array (15000):
  Average: 1.542 ms  [ 1.792, 1.417, 1.417 ]
  Sorted: ✓ Yes
Reversed Array (15000):
  Average: 1.787 ms  [ 1.646, 1.428, 2.286 ]
  Sorted: ✓ Yes
Duplicates Array (15000):
  Average: 1.996 ms  [ 2.109, 1.924, 1.956 ]
  Sorted: ✓ Yes
```

Figure 2: MergeSort (Basic) Average Times

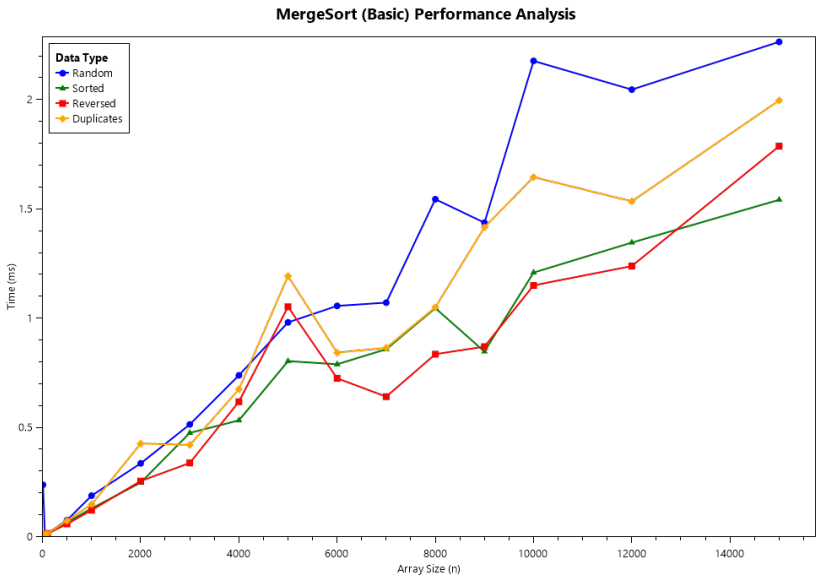

Figure 3: MergeSort (Basic) Results and Performance

MergeSort (Optimized)

The optimized version reuses a single buffer and skips the merge step if the halves are already in order.

Implementation:

```
public static class MergeSortOptimized
{
    public static void Sort(int[] arr)
    {
        if (arr == null || arr.Length < 2)
        {
            return;
        }

        int[] buffer = new int[arr.Length];
        Sort(arr, buffer, 0, arr.Length - 1);
    }

    private static void Sort(int[] arr, int[] buffer, int left, int right)
    {
        if (left >= right)
        {
            return;
        }

        int mid = left + (right - left) / 2;
        Sort(arr, buffer, left, mid);
        Sort(arr, buffer, mid + 1, right);

        if (arr[mid] <= arr[mid + 1])
        {
            return;
        }

        Merge(arr, buffer, left, mid, right);
    }

    private static void Merge(int[] arr, int[] buffer, int left, int mid, int right)
    {
        Array.Copy(arr, left, buffer, left, right - left + 1);

        int i = left;
        int j = mid + 1;
        int k = left;

        while (i <= mid && j <= right)
        {
            if (buffer[i] <= buffer[j])
            {
                arr[k++] = buffer[i++];
            }
            else
            {
                arr[k++] = buffer[j++];
            }
        }

        while (i <= mid)
        {
            arr[k++] = buffer[i++];
        }
```

Figure 4: MergeSort (Optimized) implementation in C#

Why the Optimized Version is Faster:

- **Memory Pooling:** A single auxiliary array is allocated once at the start of the sort and passed down. This completely eliminates dynamic allocation overhead.
- **Adaptive Check:** By adding an ‘if (A[mid] ¡= A[mid + 1])‘ check, the algorithm detects if the subarrays are already sorted and skips the merge phase entirely, transforming the best-case time complexity on sorted arrays closer to $O(n)$.

Basic vs Optimized: For 15,000 elements, optimized version reduced Random from 2.265 ms to 1.858 ms and Sorted from 1.542 ms to 0.174 ms.

Results:

```
--- MERGESORT (OPTIMIZED) ---
Random Array (15000):
  Average: 1.858 ms  [ 2.096, 1.735, 1.744 ]
  Sorted: ✓ Yes
Sorted Array (15000):
  Average: 0.174 ms  [ 0.175, 0.174, 0.174 ]
  Sorted: ✓ Yes
Reversed Array (15000):
  Average: 0.895 ms  [ 0.900, 0.898, 0.888 ]
  Sorted: ✓ Yes
Duplicates Array (15000):
  Average: 1.566 ms  [ 1.778, 1.619, 1.300 ]
  Sorted: ✓ Yes
```

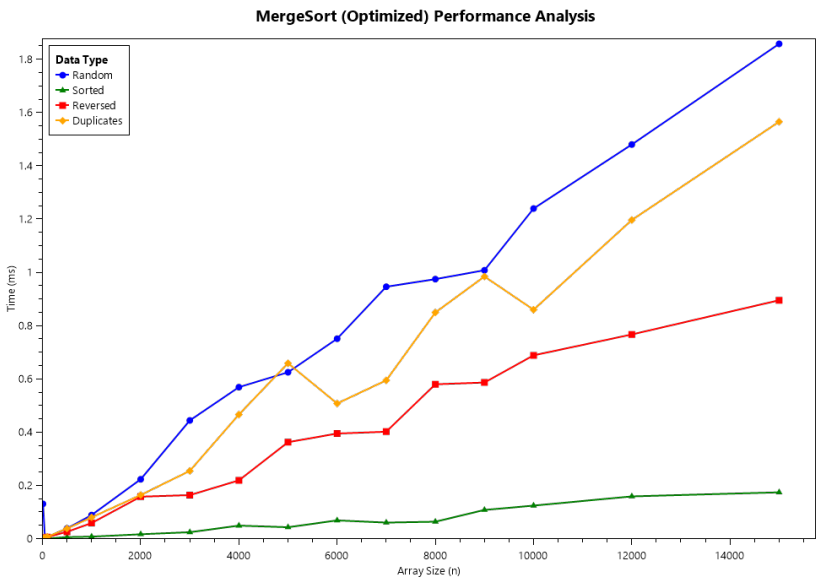

Figure 5: MergeSort (Optimized) Results and Performance

QuickSort (Basic)

The basic QuickSort uses the Lomuto partition scheme with the last element as the pivot.

Pseudocode:

Listing 2: QuickSort with Lomuto Partition

```
QuickSort(A, p, r):
    if p < r:
        q = LomutoPartition(A, p, r)

        QuickSort(A, p, q - 1)
        QuickSort(A, q + 1, r)


LomutoPartition(A, p, r):
    pivot = A[r]
    i = p - 1

    for j = p to r - 1:
        if A[j] <= pivot:
            i = i + 1
            swap A[i], A[j]

    swap A[i + 1], A[r]
    return i + 1
```

Implementation:

```csharp
public static class QuickSortBasic
{
    public static void Sort(int[] arr)
    {
        if (arr == null || arr.Length < 2)
        {
            return;
        }

        Sort(arr, 0, arr.Length - 1);
    }

    private static void Sort(int[] arr, int left, int right)
    {
        if (left < right)
        {
            int pivotIndex = LomutoPartition(arr, left, right);
            Sort(arr, left, pivotIndex - 1);
            Sort(arr, pivotIndex + 1, right);
        }
    }

    private static int LomutoPartition(int[] arr, int left, int right)
    {
        int pivot = arr[right];
        int i = left - 1;

        for (int j = left; j < right; j++)
        {
            if (arr[j] <= pivot)
            {
                i++;
                Swap(arr, i, j);
            }
        }

        Swap(arr, i + 1, right);
        return i + 1;
    }
```

Figure 6: QuickSort (Basic) implementation in C#

Complexity Facts & Performance Defects:

- **Complexity:** Best/Average Case is $O(n \log n)$. Worst Case is $O(n^2)$.
- **Why basic is slower:** Selecting the last element as a pivot (Lomuto scheme) leads to catastrophic degradation on already sorted or reverse-sorted data. It partitions an array of size n into subarrays of size $n-1$ and 0, causing the recursion tree to become a linked list and resulting in $O(n^2)$ complexity and potential stack overflow.

Results:

```
--- QUICKSORT (BASIC) ---
Random Array (15000):
  Average: 1.789 ms  [ 2.029, 1.668, 1.671 ]
  Sorted: ✓ Yes
Sorted Array (15000):
  Average: 537.719 ms  [ 537.264, 539.404, 536.489 ]
  Sorted: ✓ Yes
Reversed Array (15000):
  Average: 348.610 ms  [ 350.742, 347.041, 348.046 ]
  Sorted: ✓ Yes
Duplicates Array (15000):
  Average: 28.027 ms  [ 28.738, 27.778, 27.565 ]
  Sorted: ✓ Yes
```

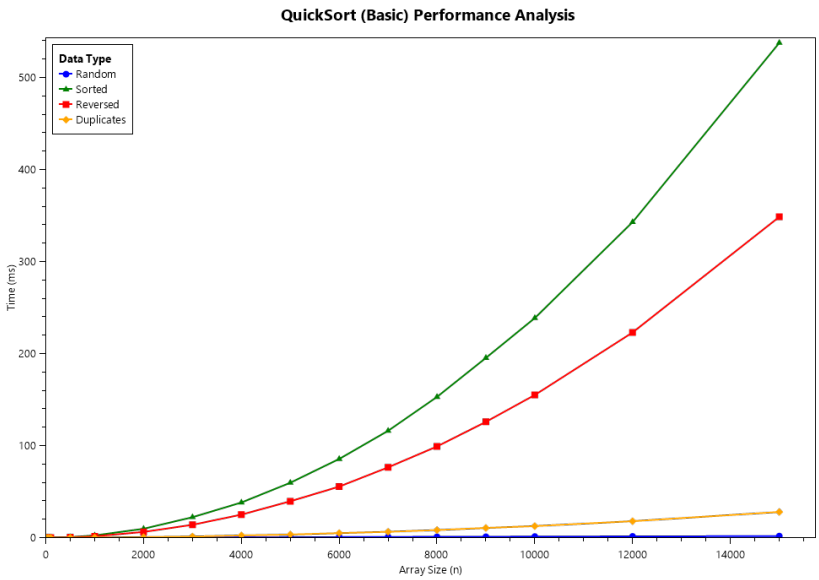

Figure 7: QuickSort (Basic) Results and Performance

QuickSort (Optimized)

The optimized version uses Hoare partitioning, median-of-three pivot selection, and tail-recursion elimination.

Pseudocode:

Listing 3: QuickSort Optimized (Hoare + Median-of-Three)

```
QuickSort(A, p, r):
    while p < r:
        q = HoarePartition(A, p, r)

        if (q - p) < (r - q):
            QuickSort(A, p, q)
            p = q + 1
        else:
            QuickSort(A, q + 1, r)
            r = q

HoarePartition(A, p, r):
    pivot = MedianOfThree(A, p, r)
    i = p - 1
    j = r + 1

    while true:
        do
            i = i + 1
        while A[i] < pivot

        do
            j = j - 1
        while A[j] > pivot

        if i >= j:
            return j
        swap A[i], A[j]

MedianOfThree(A, p, r):
    mid = (p + r) // 2

    if A[p] > A[mid]:
        swap A[p], A[mid]
    if A[p] > A[r]:
        swap A[p], A[r]
    if A[mid] > A[r]:
        swap A[mid], A[r]
    return A[mid]
```

Implementation:

```
private static int HoarePartition(int[] arr, int left, int right)
{
    int pivotIndex = MedianOfThree(arr, left, right);
    Swap(arr, left, pivotIndex);
    int pivot = arr[left];

    int i = left - 1;
    int j = right + 1;

    while (true)
    {
        do
        {
            i++;
        } while (arr[i] < pivot);

        do
        {
            j--;
        } while (arr[j] > pivot);

        if (i >= j)
        {
            return j;
        }

        Swap(arr, i, j);
    }
}

private static int MedianOfThree(int[] arr, int left, int right)
{
    int mid = left + (right - left) / 2;

    if (arr[left] > arr[mid])
    {
        Swap(arr, left, mid);
    }

    if (arr[left] > arr[right])
    {
        Swap(arr, left, right);
    }

    if (arr[mid] > arr[right])
    {
        Swap(arr, mid, right);
    }

    return mid;
}
```

Figure 8: QuickSort (Optimized) implementation in C#

Why the Optimized Version is Faster:

- **Median-of-Three:** By selecting the median of the first, middle, and last elements as the pivot, the worst-case scenario on sorted/reversed arrays is virtually eliminated, restoring $O(n \log n)$ performance.
- **Hoare Partitioning:** Performs three times fewer swaps on average than Lomuto and naturally handles duplicate elements better.

Basic vs Optimized: Sorted improved from 537.719 ms (Lomuto) to 0.949 ms (Hoare + Median-of-3), effectively eliminating the $O(n^2)$ pitfall.

Results:

```
--- QUICKSORT (OPTIMIZED) ---
Random Array (15000):
  Average: 2.038 ms  [ 2.462, 1.819, 1.834 ]
  Sorted: ✓ Yes
Sorted Array (15000):
  Average: 0.949 ms  [ 1.094, 0.739, 1.013 ]
  Sorted: ✓ Yes
Reversed Array (15000):
  Average: 1.392 ms  [ 1.652, 1.129, 1.394 ]
  Sorted: ✓ Yes
Duplicates Array (15000):
  Average: 1.410 ms  [ 1.562, 1.336, 1.331 ]
  Sorted: ✓ Yes
```

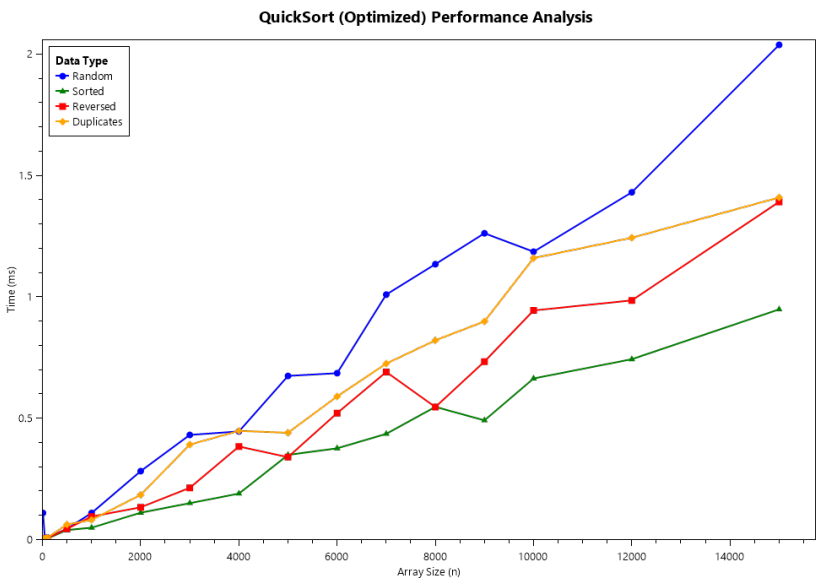

Figure 9: QuickSort (Optimized) Results and Performance

HeapSort (Basic)

HeapSort builds a Max-Heap and repeatedly extracts the root. It is an in-place $O(n \log n)$ algorithm.

Pseudocode:

Listing 4: HeapSort Complete Algorithm (Recursive Heapify)

```
HeapSort(A):
    n = length(A)

    for i = n/2 - 1 downto 0:
        Heapify(A, n, i)

    for i = n - 1 downto 1:
        swap A[0], A[i]
        Heapify(A, i, 0)


Heapify(A, n, i):
    largest = i
    left = 2*i + 1
    right = 2*i + 2

    if left < n and A[left] > A[largest]:
        largest = left

    if right < n and A[right] > A[largest]:
        largest = right

    if largest != i:
        swap A[i], A[largest]
        Heapify(A, n, largest)
```

Implementation:

```
public static class HeapSortBasic
{
    public static void Sort(int[] arr)
    {
        if (arr == null || arr.Length < 2)
        {
            return;
        }

        int n = arr.Length;

        for (int i = n / 2 - 1; i >= 0; i--)
        {
            Heapify(arr, n, i);
        }

        for (int i = n - 1; i > 0; i--)
        {
            Swap(arr, 0, i);
            Heapify(arr, i, 0);
        }
    }

    private static void Heapify(int[] arr, int heapSize, int rootIndex)
    {
        int largest = rootIndex;
        int left = 2 * rootIndex + 1;
        int right = 2 * rootIndex + 2;

        if (left < heapSize && arr[left] > arr[largest])
        {
            largest = left;
        }

        if (right < heapSize && arr[right] > arr[largest])
        {
            largest = right;
        }

        if (largest != rootIndex)
        {
            Swap(arr, rootIndex, largest);
            Heapify(arr, heapSize, largest);
        }
    }
```

Figure 10: HeapSort (Basic) implementation in C#

Complexity Facts & Performance Defects:

- **Complexity:** $O(n \log n)$ in Best, Average, and Worst cases. Space complexity is $O(1)$.
- **Why basic is slower:** The recursive implementation of 'Heapify' pushes stack frames for every level of the tree ($O(\log n)$ depth). This causes unnecessary overhead. Additionally, performing a 3-way variable swap at every heapify step increases memory writes.

Results:

```
--- HEAPSORT (BASIC) ---
Random Array (15000):
  Average: 3.644 ms  [ 3.758, 3.573, 3.601 ]
  Sorted: ✓ Yes
Sorted Array (15000):
  Average: 3.158 ms  [ 3.589, 2.932, 2.952 ]
  Sorted: ✓ Yes
Reversed Array (15000):
  Average: 2.859 ms  [ 2.880, 2.850, 2.847 ]
  Sorted: ✓ Yes
Duplicates Array (15000):
  Average: 3.325 ms  [ 3.698, 3.095, 3.181 ]
  Sorted: ✓ Yes
```

Figure 11: HeapSort (Basic) Average Times

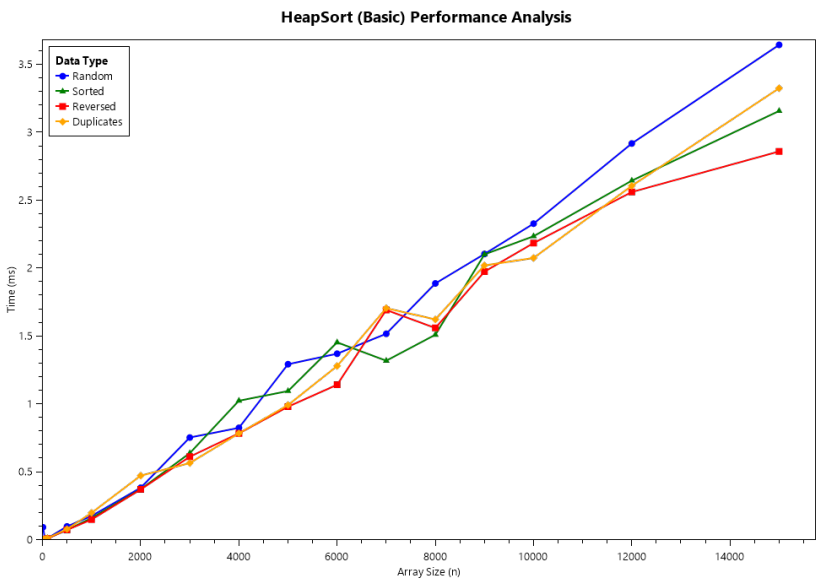

Figure 12: HeapSort (Basic) Results and Performance

HeapSort (Optimized)

The optimized version replaces recursive heapify with an iterative sift-down approach to reduce stack overhead.

Listing 5: HeapSort Optimized (Iterative Sift-Down)

```
HeapSort(A):
    if A is null or length(A) < 2:
        return

    n = length(A)

    // Build Max-Heap
    for i = (n // 2) - 1 downto 0:
        SiftDown(A, i, n)

    // Extract elements from heap
    for i = n - 1 downto 1:
        Swap(A, 0, i)
        SiftDown(A, 0, i)

SiftDown(A, root, heapSize):
    while true:
        left = 2 * root + 1

        if left >= heapSize:
            break

        right = left + 1
        swapIndex = left

        if right < heapSize and A[right] > A[left]:
            swapIndex = right

        if A[root] >= A[swapIndex]:
            break
        Swap(A, root, swapIndex)
        root = swapIndex
```

Implementation:

```
public static class HeapSortOptimized
{
    public static void Sort(int[] arr)
    {
        if (arr == null || arr.Length < 2)
        {
            return;
        }

        int n = arr.Length;

        for (int i = n / 2 - 1; i >= 0; i--)
        {
            SiftDown(arr, i, n);
        }

        for (int i = n - 1; i > 0; i--)
        {
            Swap(arr, 0, i);
            SiftDown(arr, 0, i);
        }
    }

    private static void SiftDown(int[] arr, int root, int heapSize)
    {
        while (true)
        {
            int left = 2 * root + 1;
            if (left >= heapSize)
            {
                break;
            }

            int right = left + 1;
            int swapIndex = left;

            if (right < heapSize && arr[right] > arr[left])
            {
                swapIndex = right;
            }

            if (arr[root] >= arr[swapIndex])
            {
                break;
            }

            Swap(arr, root, swapIndex);
            root = swapIndex;
        }
    }
```

Figure 13: HeapSort (Optimized) implementation in C#

Why the Optimized Version is Faster:

- **Iterative Sift-Down:** Replacing recursion with an iterative ‘while‘ loop completely eliminates call-stack overhead and prevents potential stack overflow issues on massive datasets.
- **Reduced Writes:** Instead of swapping elements continuously, the target value is held in a temporary variable, child nodes are promoted directly, and the target is inserted at the final position, reducing memory write operations by roughly two-thirds.

Results:

```
--- HEAPSORT (OPTIMIZED) ---
Random Array (15000):
  Average: 3.373 ms  [ 3.892, 3.104, 3.123 ]
  Sorted: ✓ Yes
Sorted Array (15000):
  Average: 2.611 ms  [ 2.834, 2.501, 2.499 ]
  Sorted: ✓ Yes
Reversed Array (15000):
  Average: 2.488 ms  [ 2.633, 2.422, 2.410 ]
  Sorted: ✓ Yes
Duplicates Array (15000):
  Average: 3.038 ms  [ 3.529, 2.913, 2.674 ]
  Sorted: ✓ Yes
```

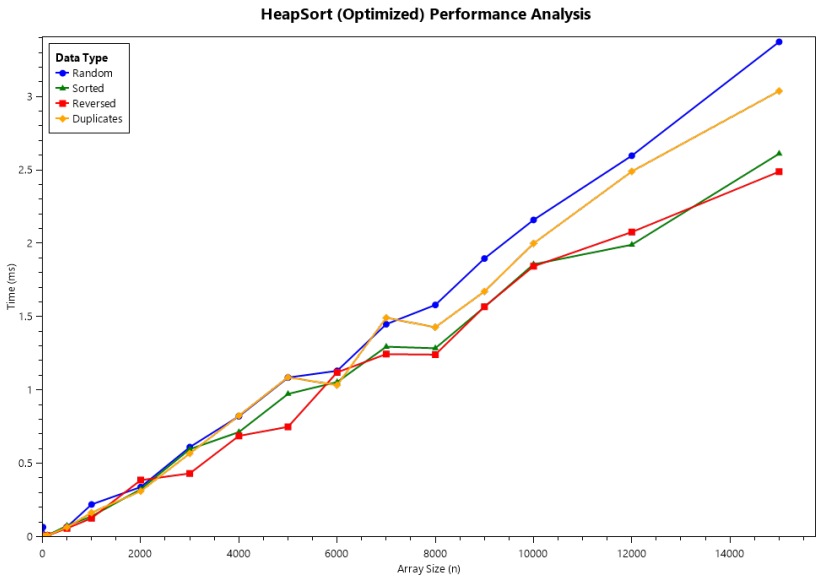

Figure 14: HeapSort (Optimized) Results and Performance

PatienceSort (Basic)

PatienceSort organizes elements into piles and merges them. The basic version uses linear scans during the merge phase.

Pseudocode:

Listing 6: PatienceSort Basic (Pile Creation + Linear Merge)

```
PatienceSort(A):
    piles = empty list

    for each element x in A:
        placed = false

        for each pile in piles:
            if x <= pile.top():
                pile.push(x)
                placed = true
                break

        if placed == false:
            create new pile with x
            add to piles

    MergePilesLinear(piles)


MergePilesLinear(piles):
    k = 0

    while piles not empty:
        minPile = pile with smallest top (linear scan)
        A[k] = minPile.pop()
        k = k + 1

        if minPile empty:
            remove minPile
```

Implementation:

```
public static void Sort(int[] arr)
{
    if (arr == null || arr.Length < 2)
    {
        return;
    }

    var piles = new List<List<int>>();

    foreach (int x in arr)
    {
        int pileIndex = FindPile(piles, x);

        if (pileIndex == -1)
        {
            piles.Add(new List<int> { x });
        }
        else
        {
            piles[pileIndex].Add(x);
        }
    }

    for (int i = 0; i < arr.Length; i++)
    {
        int minVal = int.MaxValue;
        int minPileIndex = -1;

        for (int j = 0; j < piles.Count; j++)
        {
            var pile = piles[j];
            if (pile.Count > 0)
            {
                int topVal = pile[pile.Count - 1];
                if (topVal < minVal)
                {
                    minVal = topVal;
                    minPileIndex = j;
                }
            }
        }

        arr[i] = minVal;
        var winningPile = piles[minPileIndex];
        winningPile.RemoveAt(winningPile.Count - 1);

        if (winningPile.Count == 0)
        {
            piles.RemoveAt(minPileIndex);
        }
    }
}
```

Figure 15: PatienceSort (Basic) implementation in C#

```
private static int FindPile(List<List<int>> piles, int value)
{
    int left = 0;
    int right = piles.Count - 1;
    int bestPile = -1;

    while (left <= right)
    {
        int mid = left + (right - left) / 2;
        int topCard = piles[mid][piles[mid].Count - 1];

        if (topCard >= value)
        {
            bestPile = mid;
            right = mid - 1;
        }
        else
        {
            left = mid + 1;
        }
    }

    return bestPile;
```

Figure 16: PatienceSort (Basic) FindPile function

Complexity Facts & Performance Defects:

- **Complexity:** Best case is $O(n)$ (Reversed data - 1 pile). Worst case is $O(n^2)$ due to linear merge phase on highly sorted data.
- **Why basic is slower:** On sorted data, the algorithm generates n piles. During the merge phase, finding the smallest element requires a linear scan across all n pile tops for every element. This $O(n) \times O(n)$ operation fundamentally breaks the algorithm's efficiency, creating massive execution delays.

Results:

```
--- PATIENCESORT (BASIC) ---
Random Array (15000):
  Average: 32.868 ms  [ 32.983, 32.877, 32.745 ]
  Sorted: ✓ Yes
Sorted Array (15000):
  Average: 996.896 ms  [ 995.598, 996.355, 998.735 ]
  Sorted: ✓ Yes
Reversed Array (15000):
  Average: 0.804 ms  [ 1.057, 0.678, 0.676 ]
  Sorted: ✓ Yes
Duplicates Array (15000):
  Average: 3.896 ms  [ 3.895, 3.896, 3.897 ]
  Sorted: ✓ Yes
```

Figure 17: PatienceSort (Basic) Average Times

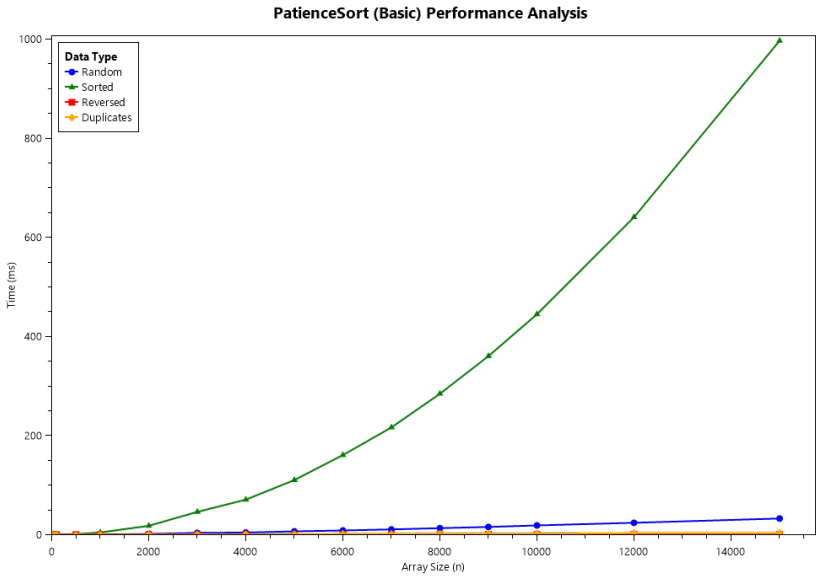

Figure 18: PatienceSort (Basic) Results and Performance

PatienceSort (Optimized)

Optimized PatienceSort uses Binary Search for pile distribution and a Priority Queue (Min-Heap) for merging piles.

Extended Pseudocode:

Listing 7: PatienceSort Optimized (Binary Search + Min-Heap Merge)

```
PatienceSortOptimized(A):
    piles = empty list

    for each element x in A:
        index = BinarySearchPile(piles, x)

        if index exists:
            piles[index].push(x)
        else:
            create new pile with x
            append to piles

    MergeUsingMinHeap(piles)


MergeUsingMinHeap(piles):
    create minHeap

    for each pile:
        insert (pile.top, pileID) into minHeap

    k = 0

    while minHeap not empty:
        node = extractMin()
        A[k] = node.value
        k = k + 1

        pop from corresponding pile

        if pile not empty:
            insert new top into minHeap
```

Implementation:

```
public static void Sort(int[] arr)
{
    if (arr == null || arr.Length < 2)
    {
        return;
    }

    var piles = new List<List<int>>();
    var pileTops = new List<int>();

    foreach (int x in arr)
    {
        int pileIndex = BinarySearchPiles(pileTops, x);

        if (pileIndex == -1)
        {
            piles.Add(new List<int> { x });
            pileTops.Add(x);
        }
        else
        {
            piles[pileIndex].Add(x);
            pileTops[pileIndex] = x;
        }
    }

    var pq = new PriorityQueue<int, int>();

    for (int i = 0; i < piles.Count; i++)
    {
        pq.Enqueue(i, piles[i][piles[i].Count - 1]);
    }

    for (int i = 0; i < arr.Length; i++)
    {
        int winningPileIndex = pq.Dequeue();
        var winningPile = piles[winningPileIndex];

        int val = winningPile[winningPile.Count - 1];
        arr[i] = val;
        winningPile.RemoveAt(winningPile.Count - 1);

        if (winningPile.Count > 0)
        {
            int newTop = winningPile[winningPile.Count - 1];
            pq.Enqueue(winningPileIndex, newTop);
        }
    }
}
```

Figure 19: PatienceSort (Optimized) implementation in C#

```
private static int BinarySearchPiles(List<int> tops, int value)
{
    int left = 0;
    int right = tops.Count - 1;
    int bestPile = -1;

    while (left <= right)
    {
        int mid = left + (right - left) / 2;

        if (tops[mid] >= value)
        {
            bestPile = mid;
            right = mid - 1;
        }
        else
        {
            left = mid + 1;
        }
    }

    return bestPile;
}
```

Figure 20: PatienceSort (Optimized) BinarySearchFiles

Why the Optimized Version is Faster:

- **Priority Queue Merge:** By feeding the top elements of each pile into a Min-Heap, extracting the minimum value drops from $O(p)$ to $O(\log p)$, where p is the number of piles. This prevents the catastrophic $O(n^2)$ worst-case and strictly guarantees $O(n \log p)$ performance.
- **Binary Search Distribution:** Using binary search to place elements into piles ensures logarithmic time per placement, handling high-volume datasets smoothly.

Results:

```
--- PATIENCESORT (OPTIMIZED) ---
Random Array (15000):
  Average: 3.024 ms  [ 2.728, 2.253, 4.091 ]
  Sorted: ✓ Yes
Sorted Array (15000):
  Average: 2.462 ms  [ 2.671, 2.358, 2.356 ]
  Sorted: ✓ Yes
Reversed Array (15000):
  Average: 0.737 ms  [ 0.956, 0.627, 0.629 ]
  Sorted: ✓ Yes
Duplicates Array (15000):
  Average: 1.405 ms  [ 1.626, 1.312, 1.277 ]
  Sorted: ✓ Yes
```

Figure 21: PatienceSort (Optimized) Average Times

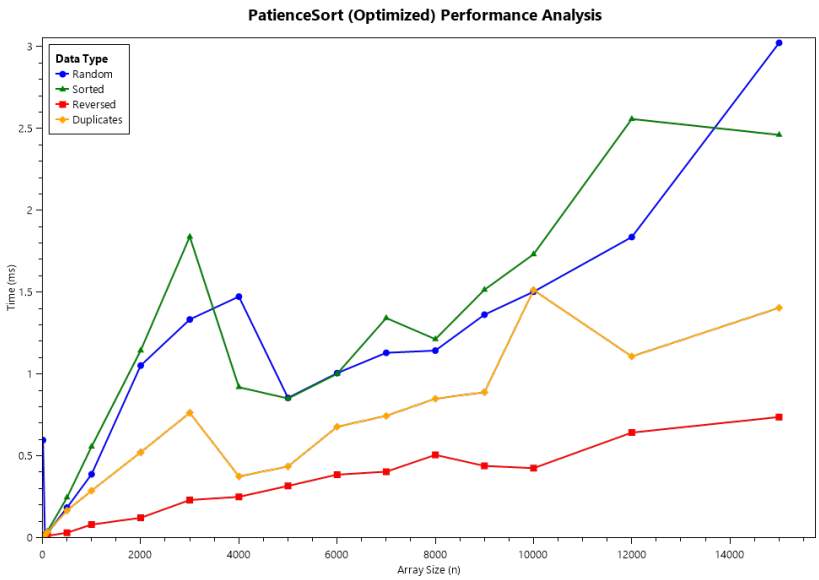

Figure 22: PatienceSort (Optimized) Results and Performance

OVERALL RESULTS

Algorithm	Random	Sorted	Reversed	Duplicates
MergeSort (Basic)	2.265 ms	1.542 ms	1.787 ms	1.996 ms
MergeSort (Optimized)	1.858 ms	0.174 ms	0.895 ms	1.566 ms
QuickSort (Basic)	1.789 ms	537.719 ms	348.610 ms	28.027 ms
QuickSort (Optimized)	2.038 ms	0.949 ms	1.392 ms	1.410 ms
HeapSort (Basic)	3.644 ms	3.158 ms	2.859 ms	3.325 ms
HeapSort (Optimized)	3.373 ms	2.611 ms	2.488 ms	3.038 ms
PatienceSort (Basic)	32.868 ms	996.896 ms	0.804 ms	3.896 ms
PatienceSort (Optimized)	3.024 ms	2.462 ms	0.737 ms	1.405 ms

Figure 23: Averages of 3 for all sorts and inputs

The overall results emphasize that each algorithm excels under specific input conditions.

MergeSort (Basic and Optimized) delivers stable and consistent performance across all datasets, with the optimized version performing exceptionally well on already Sorted data due to its merge-skip optimization.

QuickSort (Basic) performs well on Random input but collapses on Sorted and Reversed data. In contrast, **QuickSort (Optimized)** provides near-uniform high performance across all input types, making it the strongest general-purpose algorithm in this study.

HeapSort (Basic and Optimized) offers predictable and uniform performance independent of input distribution, excelling in memory efficiency due to its in-place structure.

PatienceSort (Basic) excels on Reversed data but performs poorly on Sorted input, while the optimized version successfully stabilizes performance using binary search and heap-based merging.

CONCLUSION

This empirical analysis highlights profound facts regarding algorithmic complexity, structural limitations, and the massive impact of code-level optimization. While Big-O notation provides a theoretical boundary, practical performance is dictated by constant factors, cache locality, memory management, and input structure.

The study reveals several critical insights:

- **Asymptotic Domination vs. Edge Cases:** Algorithms with an $O(n \log n)$ guarantee like MergeSort and HeapSort proved robust across all datasets. Conversely, basic QuickSort and basic PatienceSort proved dangerously fragile; Lomuto partitioning degraded to $O(n^2)$ on sorted data, taking over 537 ms, and linear-merge PatienceSort hit nearly 1,000 ms under similar conditions.

- **The Power of Structural Optimization:** Algorithm-level optimizations matter most when they resolve architectural bottlenecks. Implementing a Median-of-Three pivot and Hoare partition rescued QuickSort from stack overflows and quadratic degradation. Replacing PatienceSort's linear scan with a Priority Queue (Min-Heap) restored the algorithm's integrity, granting it reliable $O(n \log p)$ speed.

- **Memory and Resource Management:** Theoretical math ignores hardware. Basic MergeSort suffers greatly from repeated dynamic heap allocations. Pre-allocating a single auxiliary array in the optimized version brought a major constant-factor speedup, proving that memory manipulation is often more expensive than arithmetic computation. Replacing recursive calls with iterative structures, as seen in optimized HeapSort, successfully eliminated call-stack overhead.

In conclusion, performance engineering requires deep contextual awareness. MergeSort provides absolute stability but demands $O(n)$ space; HeapSort provides unparalleled memory efficiency with in-place operations; QuickSort remains the general-purpose champion but demands critical pivot safeguards; and PatienceSort demonstrates that non-standard logic excels wildly under highly specific scenarios like reversed data. Ultimately, sound algorithmic design and addressing core structural defects yield vastly superior results to simple syntax-level tuning.

REFERENCES

[1] Moisei, D. GitHub Repository. https://github.com/Moisei-D/Algorithm-Analysis